

loudnorm

Loudness Normalization. The algorithm goes like this: *Measure the integrated loudness of the source file, calculate an appropriate offset gain, and then apply makeup gain.* It's a pretty simple algorithm, but what about when there's nowhere near enough headroom for a simple upwards linear gain adjustment? And how should we handle the loudness normalization of livestreams?

Input Integrated:	-27.6 LUFS
Input True Peak:	-4.5 dBTP
Input LRA:	18.1 LU
Input Threshold:	-39.2 LUFS

Consider the source above. If we want to loudness normalize this audio to `-16.0 LUFS IL` with a maximum true peak of `-1.5 dBTP`, values which are within the AES streaming loudness recommendation, we're going to need a dynamic, *non-linear* algorithm.

I spent the last few months designing a good sounding dynamic loudness normalization algorithm to solve this problem, which has been implemented as an audio filter for FFmpeg. The loudnorm filter consists of a specially designed loudness-tuned AGC followed by a transparent look-ahead true peak limiter.

Loudness AGC

The EBU R128 loudness standard gives us our integrated loudness definition: *The measurement input to which the gating threshold is applied is the loudness of the 400 ms blocks with a constant overlap between consecutive gating blocks of 75%.* The loudnorm AGC uses this definition to calculate an appropriate correction envelope.

A good AGC needs to have an envelope that is as flat and slow moving as possible, yet also able to deal with sudden changes in program loudness. In this AGC, local dynamics are maintained by specifying a target LRA, and only adjusting when loudness exceeds this range (in either direction.) The envelope is smoothed with a Gaussian kernel and linearly interpolated, which allows it to deal with sudden rapid changes in loudness while also still being able to honor local dynamics.

A good AGC also needs to be able to differentiate “silence” from signal. This is where many AGCs fail, either neglecting this completely, or specifying a “silence” gate with some arbitrary value. Most of us have probably heard some ugly dynamics processor ramp up the noise floor a few seconds after an orchestra finishes its last note. Fortunately, the EBU R128 standard also provides us with a *relative gating threshold* which puts this “silence” threshold in perceptual context within the program, shifting as it goes along.

True Peak Limiter

The AGC sounds great, but it doesn’t protect the signal from exceeding a maximum true peak value. For this, we need a true peak limiter. To accurately detect **true peaks**, the audio is upsampled to 192 kHz. The limiter uses a 100 ms look-ahead, which allows it to calculate and use the absolute minimum amount of gain reduction required, as well as react before each peak takes place. This also allows the limiter to sustain instead of releasing if it detects another peak in the same neighborhood. This results in a transparent limiter with no pumping, even during periods of sustained peaks. In testing there is no distortion, even with several dB of reduction.

Using the FFmpeg Filter

The loudnorm filter can either operate in *single-pass* or *dual-pass* mode. Single-pass mode is useful for livestreams, and dual-pass mode is useful for file based normalization. For single-pass mode, just supply your target loudness values when calling FFmpeg.

```
ffmpeg -i in.wav -af loudnorm=I=-16:TP=-1.5:LRA=11 -ar 48k out.wav
```

Dual-pass mode requires two calls to the loudnorm filter, the first of which is for measurement. This command will print loudness stats to stderr as JSON, and create no output file. Of course, dual-pass normalization can be easily scripted.

```
ffmpeg -i in.wav -af loudnorm=I=-16:TP=-1.5:LRA=11:print_format=json -f null -
```

```
{
    "input_i" : "-27.61",
    "input_tp" : "-4.47",
    "input_lra" : "18.06",
    "input_thresh" : "-39.20",
    "output_i" : "-16.58",
    "output_tp" : "-1.50",
    "output_lra" : "14.78",
```

```
"output_thresh" : "-27.71",
"normalization_type" : "dynamic",
"target_offset" : "0.58"
}
```

We can then use these stats to call FFmpeg for a second time. Supplying the filter with these stats allow it to make more intelligent normalization decisions, as well as normalize linearly if possible. This second pass creates the loudness normalized output file, and prints a human-readable stats summary to stderr.

```
ffmpeg -i in.wav -af loudnorm=I=-16:TP=-1.5:LRA=11:measured_I=-27.61:measured_L
```

```
Input Integrated:    -27.5 LUFS
Input True Peak:     -4.5 dBTP
Input LRA:           18.1 LU
Input Threshold:     -39.2 LUFS

Output Integrated:   -16.0 LUFS
Output True Peak:    -1.5 dBTP
Output LRA:          14.6 LU
Output Threshold:    -27.2 LUFS

Normalization Type:  Dynamic
Target Offset:       +0.0 LU
```

A few final notes about this filter. Although it can target specific LRAs, this value is not guaranteed, it should however be accurate after sufficient integration time. Also, output integrated loudness may be off by a small amount if integration time is not sufficient. For precise integrated loudness, always use dual-pass mode and supply the offset parameter. This linear offset gain is pre-limiter, so it will not affect your true peak value. Maximum true peak is always guaranteed. Finally, for true peak limiting, this filter upsamples to 192 kHz. It is up to you to downsample to an appropriate sampling rate. The two examples above show how to create 48 kHz output files.

Listening Test

- loudnorm_before.mp3
- loudnorm_after.mp3

```
Input Integrated:    -24.1 LUFS
```

Input True Peak:	-6.0 dBTP
Input LRA:	10.2 LU
Input Threshold:	-34.6 LUFS

Output Integrated:	-16.0 LUFS
Output True Peak:	-1.5 dBTP
Output LRA:	8.8 LU
Output Threshold:	-26.4 LUFS

Normalization Type:	Dynamic
Target Offset:	+0.0 LU

signal hacking and low-level programming
k@ylo.ph